THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 5

Rohan garg(1024150187)

1.

Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

main.cpp

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    void showBase() {
        cout << "This is Base Class" << endl;
    }
};

class Derived : public Base {
public:
    void showDerived() {
        cout << "This is Derived Class" << endl;
    }
};

int main() {
    Base b;
    Derived d;

    b.showBase();
    d.showBase();   // inherited
    d.showDerived();

    return 0;
}
```

Output

```
This is Base Class
This is Base Class
This is Derived Class


=== Code Execution Successful ===
```

2.

Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

main.cpp

```cpp
#include <iostream>
using namespace std;

class Base {
protected:
    int x;
public:
    void setX(int a) {
        x = a;
    }
};

class Derived : public Base {
public:
    void display() {
        cout << "Value of x: " << x << endl;
    }
};

int main() {
    Derived d;
    d.setX(10);
    d.display();

    return 0;
}
```

Output

```
Value of x: 10


=== Code Execution Successful ===
```

3.

Most of the time we use public mode of inheritance, for example:

```
class Derived : public Base {}
```

Try protected and private access modifiers to understand the difference of various modes of inheritance.

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    int x;
};

class PublicDerived : public Base {
public:
    void show() { cout << x << endl; }
};

class ProtectedDerived : protected Base {
public:
    void set() { x = 20; cout << x << endl; }
};

class PrivateDerived : private Base {
public:
    void set() { x = 30; cout << x << endl; }
};
```

```cpp
int main() {
    PublicDerived p;
    p.x = 10;
    p.show();

    ProtectedDerived pr;
    pr.set();

    PrivateDerived pv;
    pv.set();

    return 0;
}
```

```
10
20
30


=== Code Execution Successful ===
```

4.

Various types of inheritances are as shown. Write small C++ codes for each inheritance type:

- Single inheritance

main.cpp

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "Class A" << endl; }
};

class B : public A {
public:
    void showB() { cout << "Class B" << endl; }
};

int main() {
    B obj;
    obj.showA();
    obj.showB();
}
```

Output

```
Class A
Class B


=== Code Execution Successful ===
```

- Multiple inheritance

main.cpp

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "A" << endl; }
};

class B {
public:
    void showB() { cout << "B" << endl; }
};

class C : public A, public B {
};

int main() {
    C obj;
    obj.showA();
    obj.showB();
}
```

Output

```
A
B


=== Code Execution Successful ===
```

- Hierarchical inheritance

main.cpp

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "Base A" << endl; }
};

class B : public A {
};

class C : public A {
};

int main() {
    B b;
    C c;

    b.showA();
    c.showA();
}
```

Output

```
Base A
Base A


=== Code Execution Successful ===
```

- Multilevel inheritance

main.cpp

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "A" << endl; }
};

class B : public A {
};

class C : public B {
};

int main() {
    C obj;
    obj.showA();
}
```

Output

```
A


=== Code Execution Successful ===
```

- Hybrid inheritance

main.cpp

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "A" << endl; }
};

class B : public A {
};

class C : public A {
};

class D : public B, public C {
};

int main() {
    D obj;
    obj.B::showA();  // avoid ambiguity
    obj.C::showA();
}
```

Output

```
A
A


=== Code Execution Successful ===
```

---

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

main.cpp

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base Constructor" << endl;
    }
    ~Base() {
        cout << "Base Destructor" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived Constructor" << endl;
    }
    ~Derived() {
        cout << "Derived Destructor" << endl;
    }
};

int main() {
    Derived d;
    return 0;
}
```

Output

```
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor


=== Code Execution Successful ===
```

6.

Implement a base class `Book` with attributes title, author, and price.
Create a derived class `Textbook` that adds subject.
Demonstrate how single inheritance is used to manage data.

```cpp
#include <iostream>
using namespace std;

class Book {
protected:
    string title, author;
    float price;
public:
    void input() {
        cin >> title >> author >> price;
    }
};
class Textbook : public Book {
    string subject;
public:
    void getData() {
        input();
        cin >> subject;
    }
    void display() {
        cout << title << " " << author << " " << price << " " << subject << endl;
    }
};
int main() {
    Textbook t;
    t.getData();
    t.display();
}
```

7.

A software company is creating a program to simulate a car's dashboard.

- Speed → Speedometer class
- Fuel level → FuelGauge class
- Temperature → Thermometer class

Create a `CarDashboard` class using multiple inheritance to display combined information.

main.cpp

Run

```cpp
#include <iostream>
using namespace std;

class Speedometer {
public:
    int speed = 60;
};

class FuelGauge {
public:
    int fuel = 50;
};

class Thermometer {
public:
    int temp = 90;
};

class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    void display() {
        cout << "Speed: " << speed << endl;
        cout << "Fuel: " << fuel << endl;
        cout << "Temperature: " << temp << endl;
    }
};

int main() {
    CarDashboard c;
    c.display();
}
```

Output

Clear

```
Speed: 60
Fuel: 50
Temperature: 90


=== Code Execution Successful ===
```

8.

Create a library system:

- Base class: `LibraryUser` (name, ID, contact)
- Derived classes: `Student` (grade level), `Teacher` (department)

Demonstrate hierarchical inheritance.

main.cpp

```cpp
#include <iostream>
using namespace std;

class LibraryUser {
protected:
    string name;
public:
    void input() { cin >> name; }
};

class Student : public LibraryUser {
    int grade;
public:
    void get() {
        input();
        cin >> grade;
    }
    void show() {
        cout << name << " " << grade << endl;
    }
};

class Teacher : public LibraryUser {
    string dept;
public:
    void get() {
        input();
        cin >> dept;
    }
    void show() {
        cout << name << " " << dept << endl;
    }
};

int main() {
    Student s;
    Teacher t;

    s.get();
    t.get();

    s.show();
    t.show();
}
```

Output

```
Aadhya
10
Manya
9
Aadhya 10
Manya 9


=== Code Execution Successful ===
```

9.

Vehicle management system:

- Base class: `Vehicle` (make, model, year)
- Derived: `Truck` (load_capacity)
- Further derived: `RefrigeratedTruck` (temperature_control)

Demonstrate multilevel inheritance.

```cpp
#include <iostream>
using namespace std;

class Vehicle {
protected:
    string make;
public:
    void input() { cin >> make; }
};

class Truck : public Vehicle {
protected:
    int load;
public:
    void get() {
        input();
        cin >> load;
    }
};

class RefrigeratedTruck : public Truck {
    int temp;
public:
    void getData() {
        get();
        cin >> temp;
    }
    void show() {
        cout << make << " " << load << " " << temp << endl;
    }
};

int main() {
    RefrigeratedTruck r;
    r.getData();
    r.show();
}
```

```
Car
100
5000
Car 100 5000


=== Code Execution Successful ===
```

10.

Academic system:

Base class: `Person` (name, address)

Derived: `Staff` (employee_id, department)

Derived: `Student` (student_id, grade)

Hybrid: `TeachingAssistant` inherits from both Staff and Student

Demonstrate hybrid inheritance.

```cpp
#include <iostream>
using namespace std;

class Person {
protected:
    string name;
public:
    void input() { cin >> name; }
};

class Staff : public Person {
protected:
    int emp_id;
public:
    void getStaff() {
        input();
        cin >> emp_id;
    }
};

class Student : public Person {
protected:
    int student_id;
public:
    void getStudent() {
        input();
        cin >> student_id;
    }
};
```

```cpp
class TeachingAssistant : public Staff, public Student {
public:
    void display() {
        cout << "TA Info:" << endl;
        cout << "Staff ID: " << emp_id << endl;
        cout << "Student ID: " << student_id << endl;
    }
};

int main() {
    TeachingAssistant ta;

    ta.getStaff();
    ta.getStudent();
    ta.display();

    return 0;
}
```

```
212
100
100
212
TA Info:
Staff ID: 100
Student ID: 212


=== Code Execution Successful ===
```